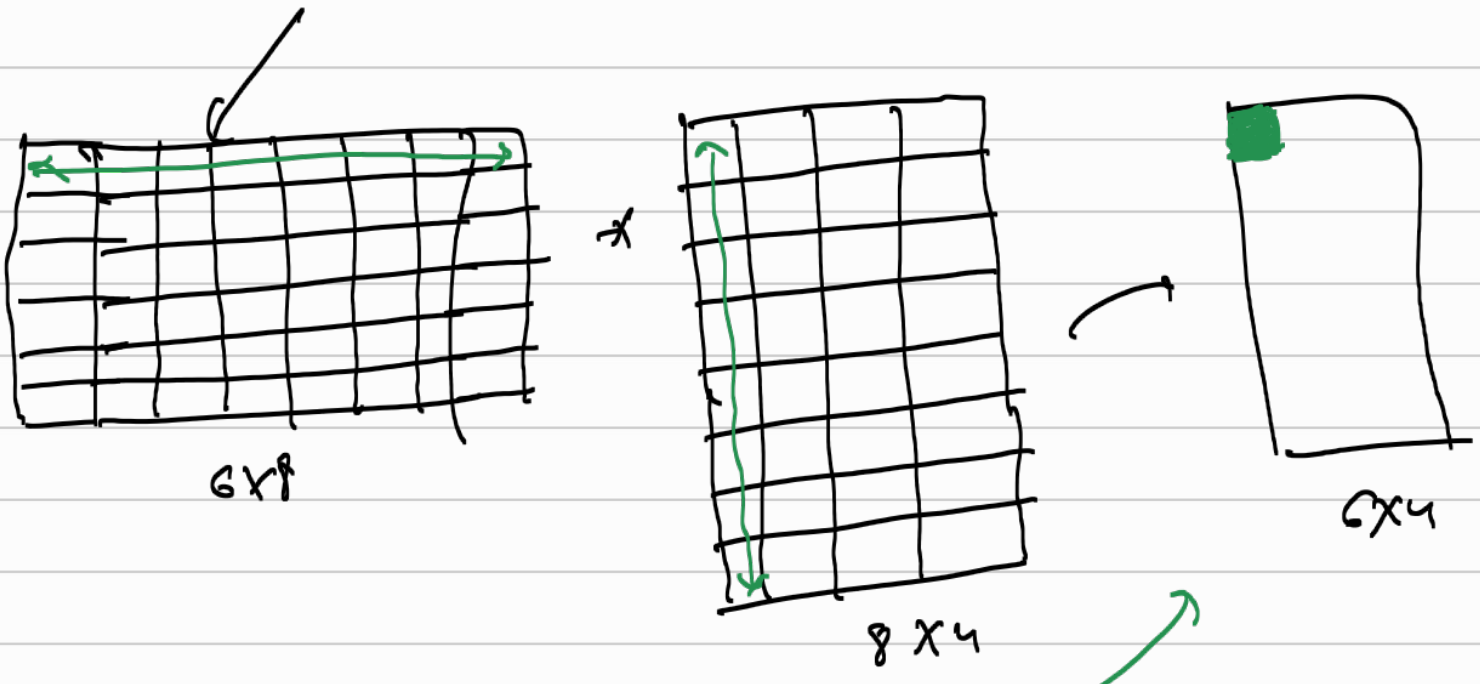
Q3 → Triton Naive Matrix multiplication:-

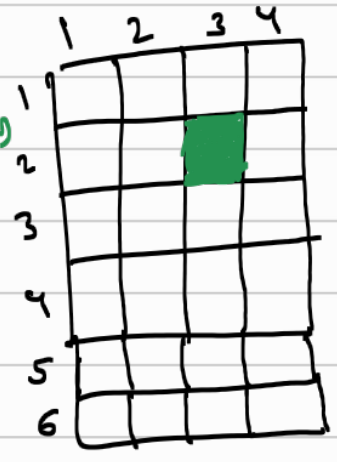
Ex:-

$A_{6\times8} * B_{8\times4} \longrightarrow C_{6\times4}$



→ $A_{1,j} * B_{j,1} \longrightarrow C_{1\times1}$

→ $A_{2,j} * B_{j,3} \longrightarrow C_{2\times3}$



Naive method :-

```
def naive_method (A, B) {
    M, K = A.shape
    K2, N = B.shape.
    assert K == K2
    C = torch.zeros(M, N)

    def dot(V1, V2):
        acc = 0
        for p, d in zip(V1, V2):
            acc += p * d;
        return acc.

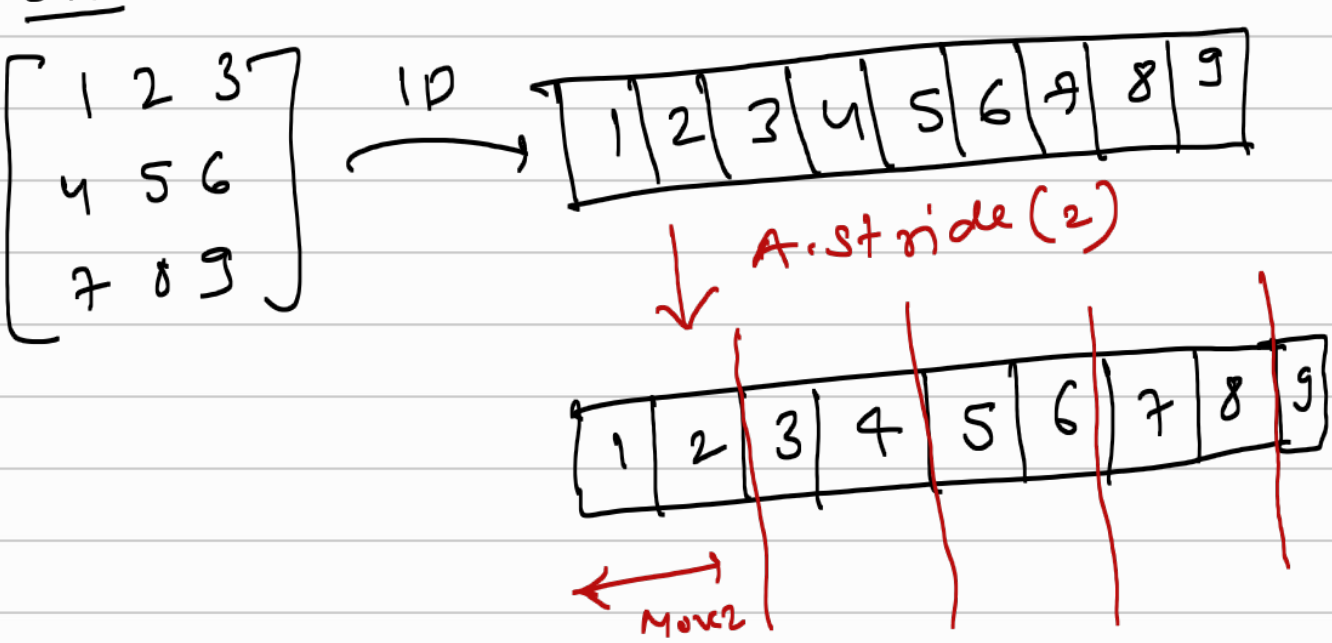
    num_pids = list(range(N * M)):
    for i in num_pids:
        m = i // N ← row
        n = i % N ← col
        C[m, n] = dot(A[m, :], B[:, n])

    return C.
```

Matrix-Mat with Triton kernal :-

→ stride(2)

Ex:-



→ M, K = A.shape

→ K2, N = B.shape.

→ assert K == K2

→ C = torch.zeros((M, N),)

→ grid = (M*N,)

→ naive_matmul_kernel[grid](

A, B, C,

M, N, K,

A.stride(0), A.stride(1)

B.stride(0), B.stride(1)

C.stride(0), C.stride(1)).

→ return C.

```
@triton.jit
def naive_matmul_kernel(A_ptr, B_ptr, C_ptr,
                        M, N, K,
                        stride_am, stride_ak
                        stride_bk, stride_bn
                        stride_cm, stride_cn):
```

which pid

Example:

C →

0	1	2	3
4	5	6	7
8	9	10	11

← Total pid = 11
then pid will be anywhere b/w 0 to 11

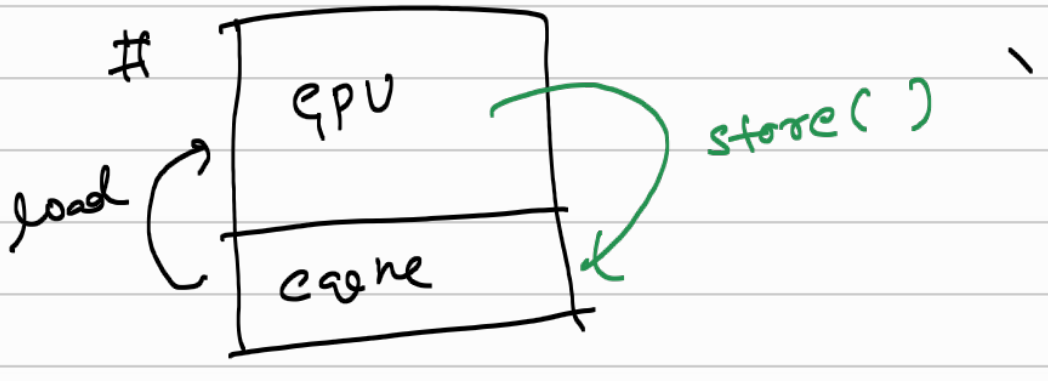
```
pid = tl.program_id(axis=0)
```

same again which row and col.

m = idx // N, n = idx % N

```
m = pid // N
n = pid % N
```

```
if (m >= M) or (n >= N):
    return
```



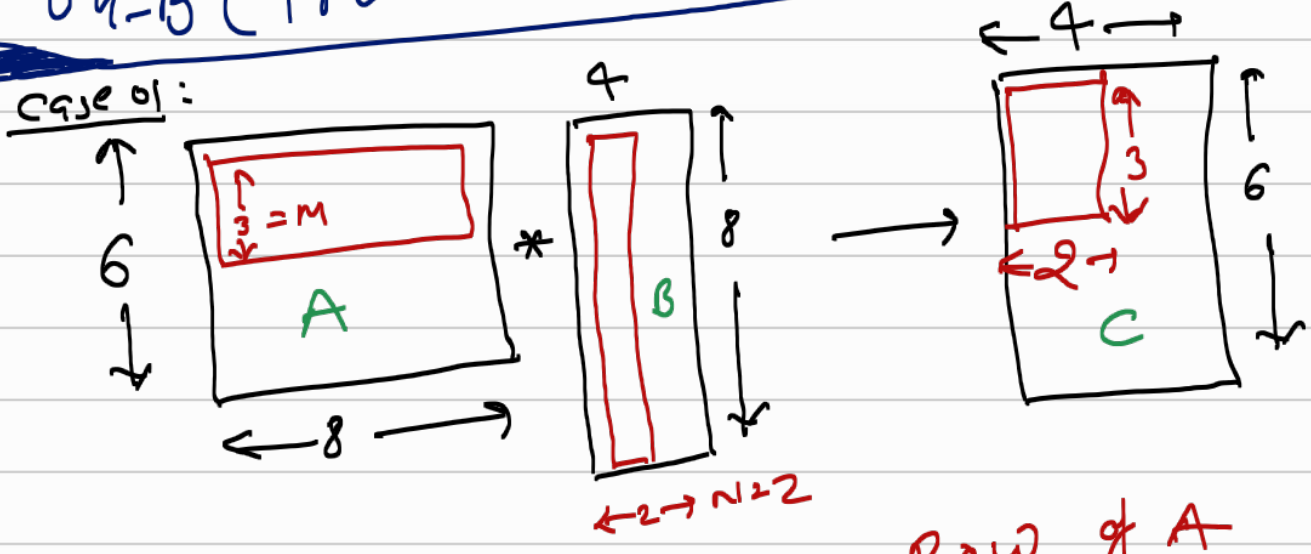
```
acc = 0.0
for k in range(0, K):
    a_val = tl.load(A_ptr + m * stride_am + k * stride_ak)
    b_val = tl.load(B_ptr + n * stride_bn + k * stride_bk)

    acc += a_val * b_val

tl.store(C_ptr + m * stride_cm + n * stride_cn, acc)
```

04-B (Triton Blocked Matrix multiplication) :-

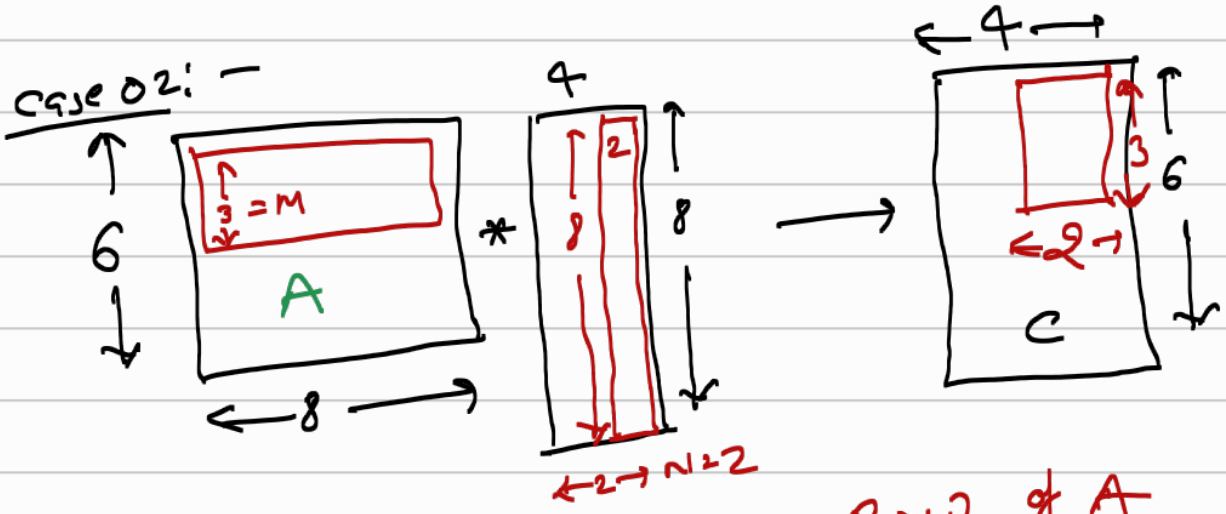
Case 01 :



Block size (M) = 3 ← Row of A

Block size (N) = 2 ← Col of B.

Case 02: –



Block size (M) = 3 ← Row of A
Block size (N) = 2 ← Col of B.

Same way (M = 3) move A and N move 2 again;

$\{A, B\}(S_r, S_{col}, E_r, E_{col})$

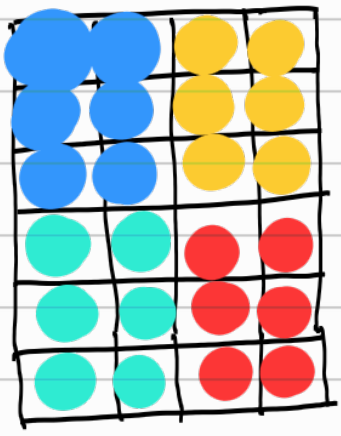
$\{A, B\}((0,0) \xrightarrow{A} (2,7) \mid (0,0) \xrightarrow{B} (1,1))$ ← Rearrange

move (N = 2)

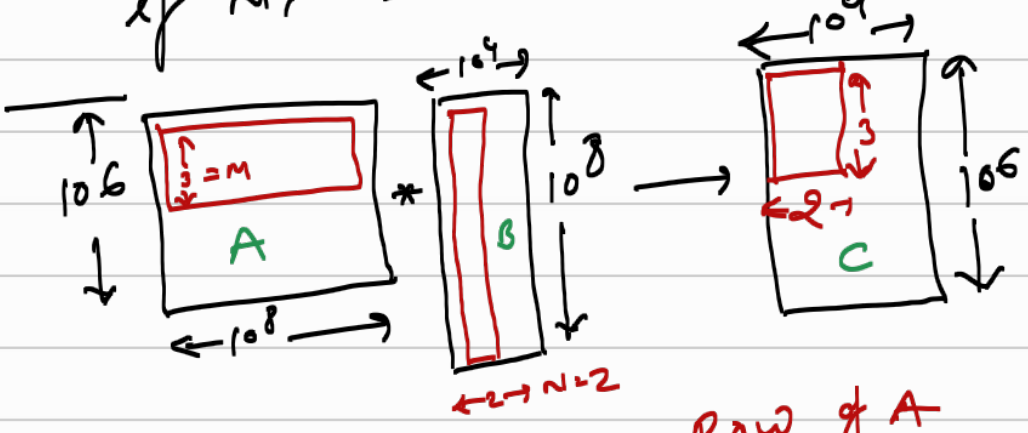
$\{A, B\}((0,0) \xrightarrow{A} (2,7) \mid (0,2) \rightarrow (8,3))$

→ Same way of other Branch of Tree.

→ then final o/p is like this way



→ now prev case we take full of Row and full of col, what happen if $N, n \simeq 10^6$ then.



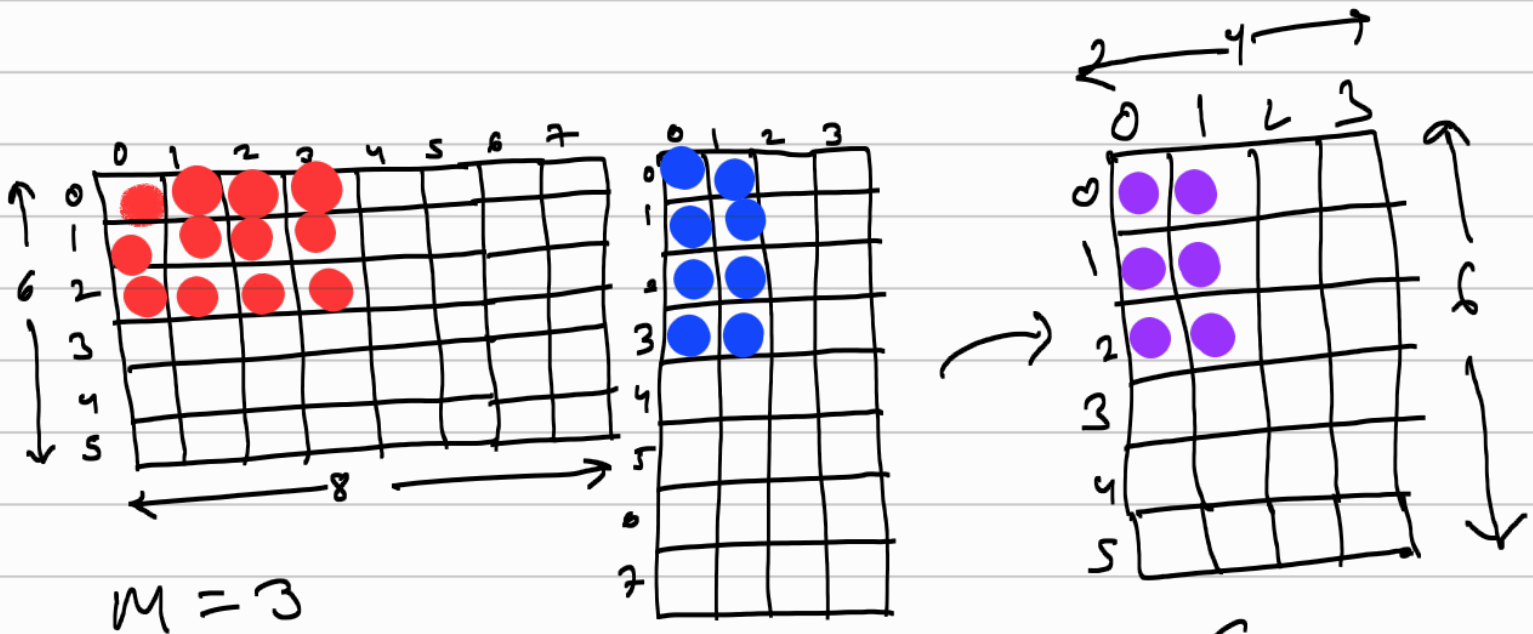
Block size (M) = 3 ← Row of A
Block size (N) = 2 ← Col of B.

then:

$$A_{3 \times 10^8} * B_{(10^8 \times 2)} \longrightarrow C_{(3 \times 2)}$$

→ due to large compute it's take lot of memory. then add one more Block.

Block size (M) = 3 ← Row of A
Block size (N) = 2 ← Col of B.
Block size (K) = 4 ← end of Row and Col.



$M = 3$

$N = 2$

$K = 4$ → $A_{3\times 8} * B_{(8\times 2)} \rightarrow C_{(3\times 2)}$

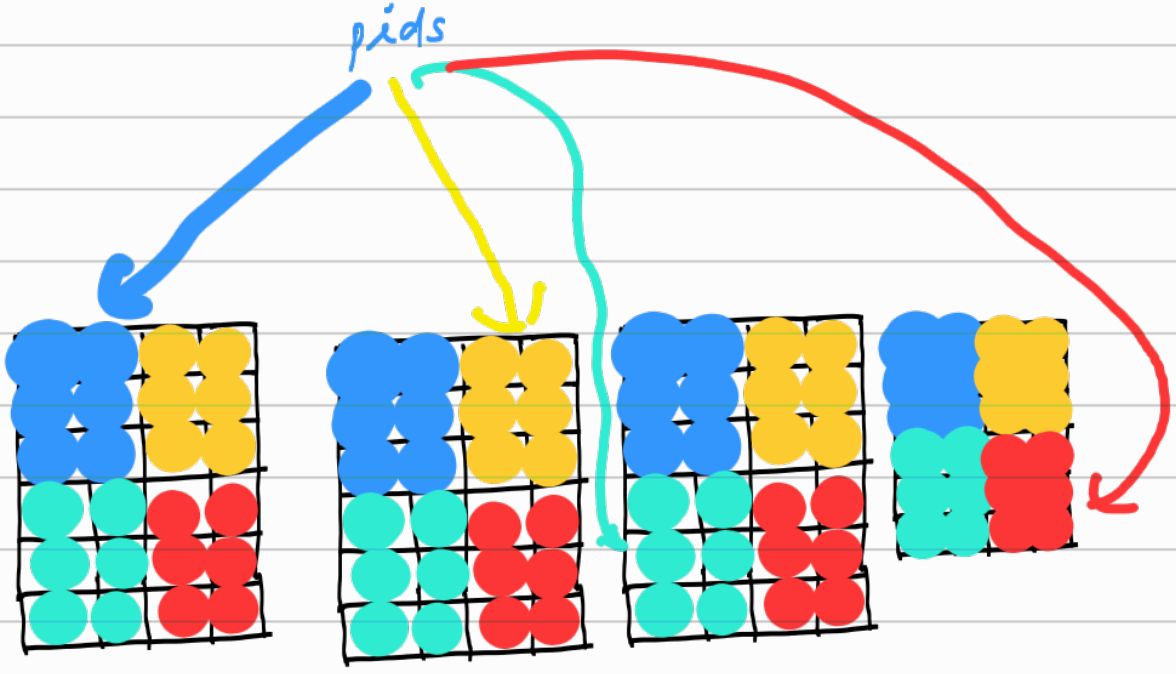
$A_{(3\times 4)} * B_{(4\times 2)} \rightarrow C_{(3\times 2)}$

→ Method 01:-

```
→ def BlockedMatmal(A, B, blocksize_M, blocksize_N):
      M, K = A.shape
      K, N = B.shape
      C = torch.zeros(M, N)
      assert M % BlockSize_M == 0
      assert N % BlockSize_N == 0

      num_Block_row = M // BlockSize_M
      num_Block_col = N // BlockSize_N
      total Block = num_block_cols * num_block_row
      num_pids = list(range(total_Blocks))
```

Ex:- each pids responsible for each Block



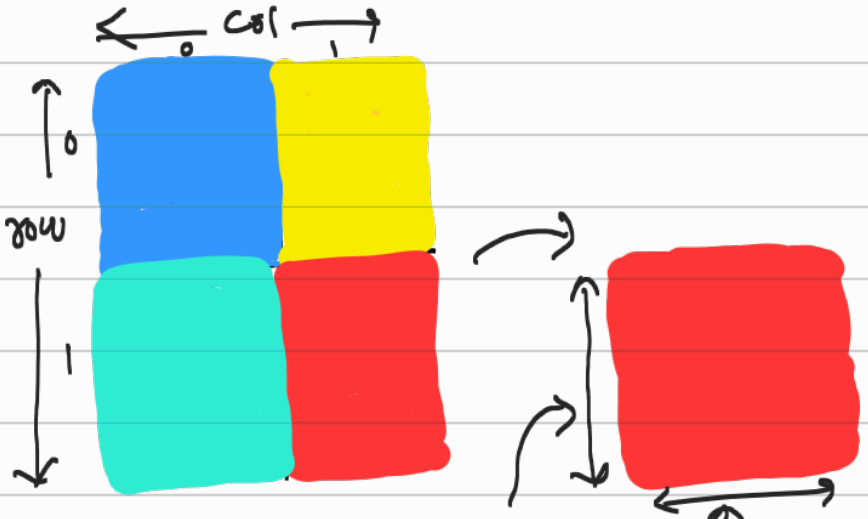
for i in pids:

block_m = i // num_block_cols

block_n = i % num_block_cols

_m → which block of rows are we on?

_n → which block of col are we on?



m = block_m * Block_SIZE_M

n = block_n * Block_SIZE_N

block_A = A[m : m + BlockSize_M, :]

block_B = B[:, n : n + BlockSize_N]

C[m:m+BlockSize_M, n:n+BlockSize_N] = blockA @ block_B.

return C.

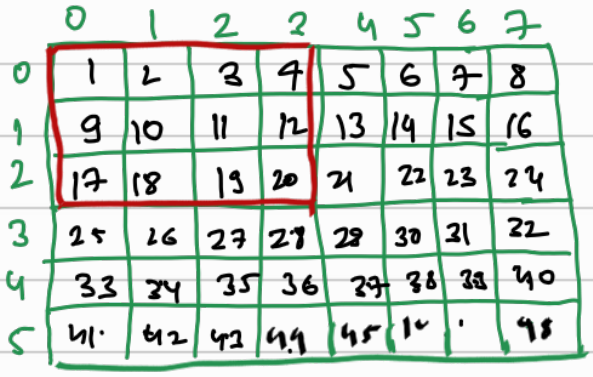
Method 02:- Triton kernal:-

```
def blocked_matmul_kernel(
    A_ptr, B_ptr, C_ptr,
    M , N , K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    BLOCK_SIZE_M : tl.constexpr,
    BLOCK_SIZE_N : tl.constexpr,
    BLOCK_SIZE_K : tl.constexpr ) :
```

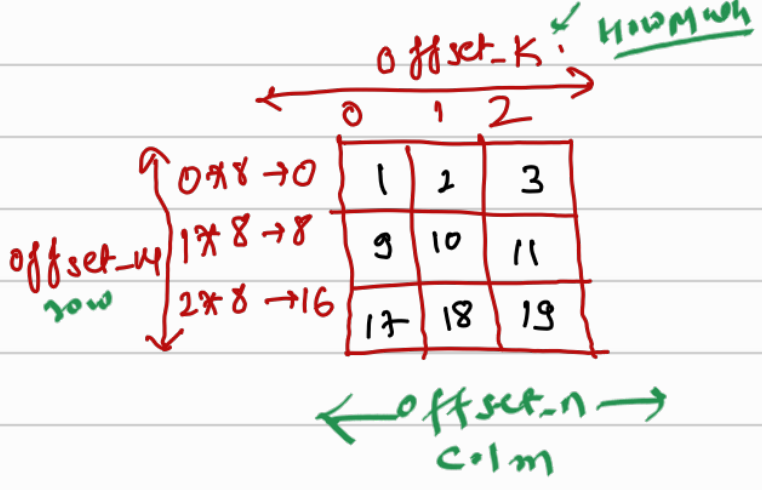
FOR Dynamic

```
pid = tl.program_id (axis = 0)
# Get how ma  columns of Blocks we have ###
num_block_cols = tl.cdiv(N, BLOCK_SIZE_N)
```

```
block_m = pid // num_block_cols
block_n = pid % num_block_cols
```



BLOCK_size_M : 3
Block_size_K. : 4




```
offset_m = block_m * BLOCK_SIZE_M + tl.arange(0, Block_size_M)
offset_n = block_n * Block_size_N + tl.arange(0, Block_size_N)
```

Ex:

```
offset_m = [0] + [0,1,2] → [0,1,2]
         = [1*3] + [0,1,2] → [3,4,5]
```

} goto..

offset_m = [0] + [0,1,2,3] → [0,1,2,3]
= [1x4] + [0,1,2,3] → [4,5,6,7]

offset_k = tl.arange(0, BLOCK_SIZE_K)

a_ptr = A_ptr + (offset_m[:, None] * stride_am + offset_k[None, :] * stride_ak)

b_ptr = B_ptr + (

)

None used for add DIM/AXIS

offset_m ∈ (BLOCK_SIZE_M, 1)

offset_k ∈ (BLOCK_SIZE_N, 1)

accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N),
dtype = tl.float32)

for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):

a_mask = (offs_m[:, None] < M) and (offs_k[:, None] < K - k * BLOCK_SIZE_K)

a = tl.load(a_ptrs, mask = a_mask, other = 0.0)

b_mask = ( ) and ( )

b = tl.load(b_ptrs, mask = b_mask, other = 0.0)

accumulator += tl.dot(a, b)

a_ptr += BLOCK_SIZE_K * stride_ak

b_ptr += BLOCK_SIZE_K * stride_bk.

c = accumulator.to(A_ptr.dtype.element_ty)

c_offsets = stride_cm * offs_m[:, None] + stride_cn * offs_n[None, :]

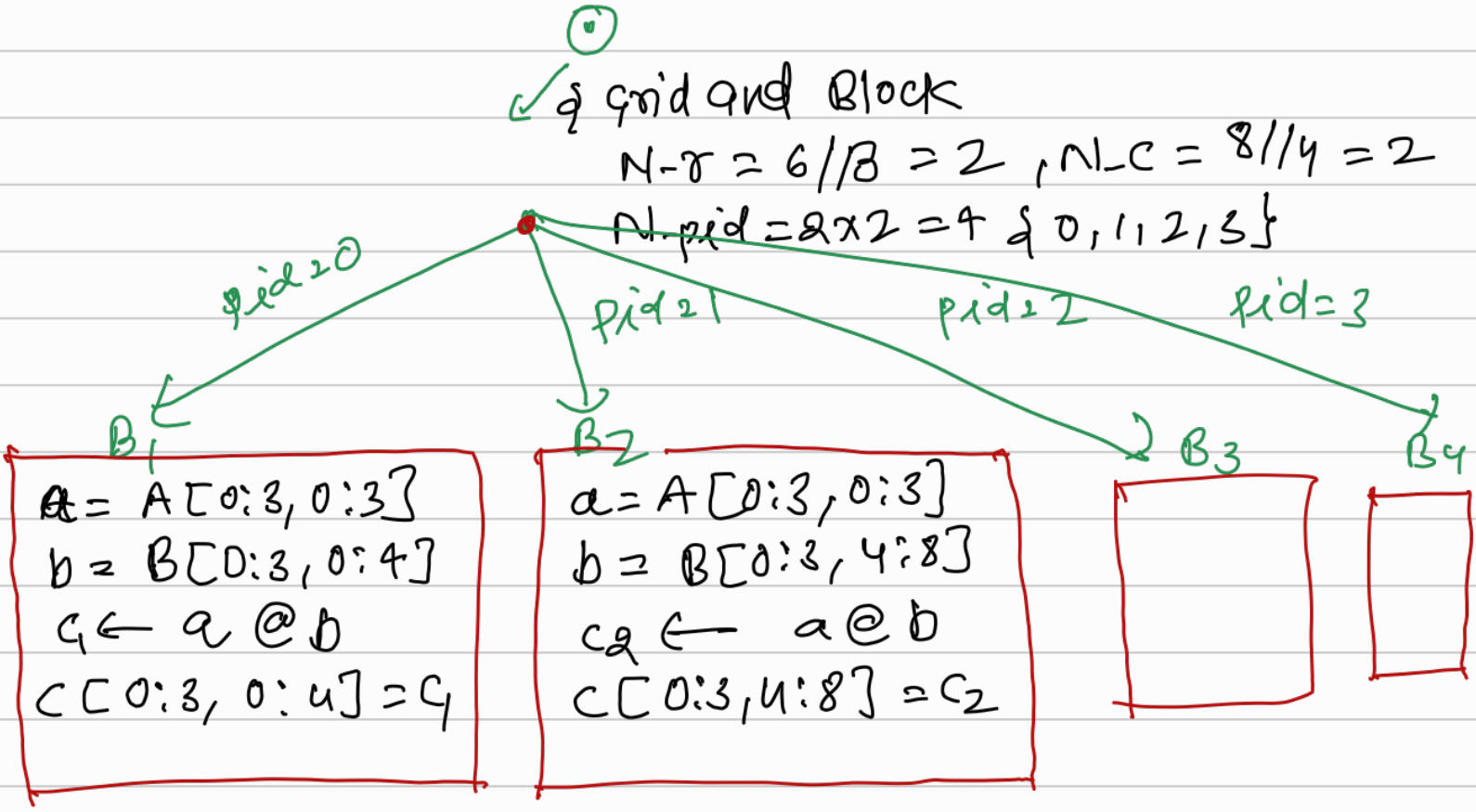
c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)

tl.store(c_ptr + c_offsets, c, mask = c_mask)

```
def blocked_matmul(a,b):
    M,K = a.shape
    K2,N = b.shape
    assert K == K2
    c = torch.empty((M,N), device = ---, ---)
    grid = lambda meta: (triton.cdiv(M, meta["BLOCK_SIZE"])
                         * triton.cdiv(N, meta[BLOCK_SIZE_N]))
    blocked_matmul_kernel[grid](
        a,b,c,
        M,N,K,
        a.stride(0), a.stride(1),
        b.stride(0), b.stride(1),
        c.stride(0), b.stride(1)
    )
    return c
```

Method 01:- ex: A (6X3) B(3X8) = C(6X8)

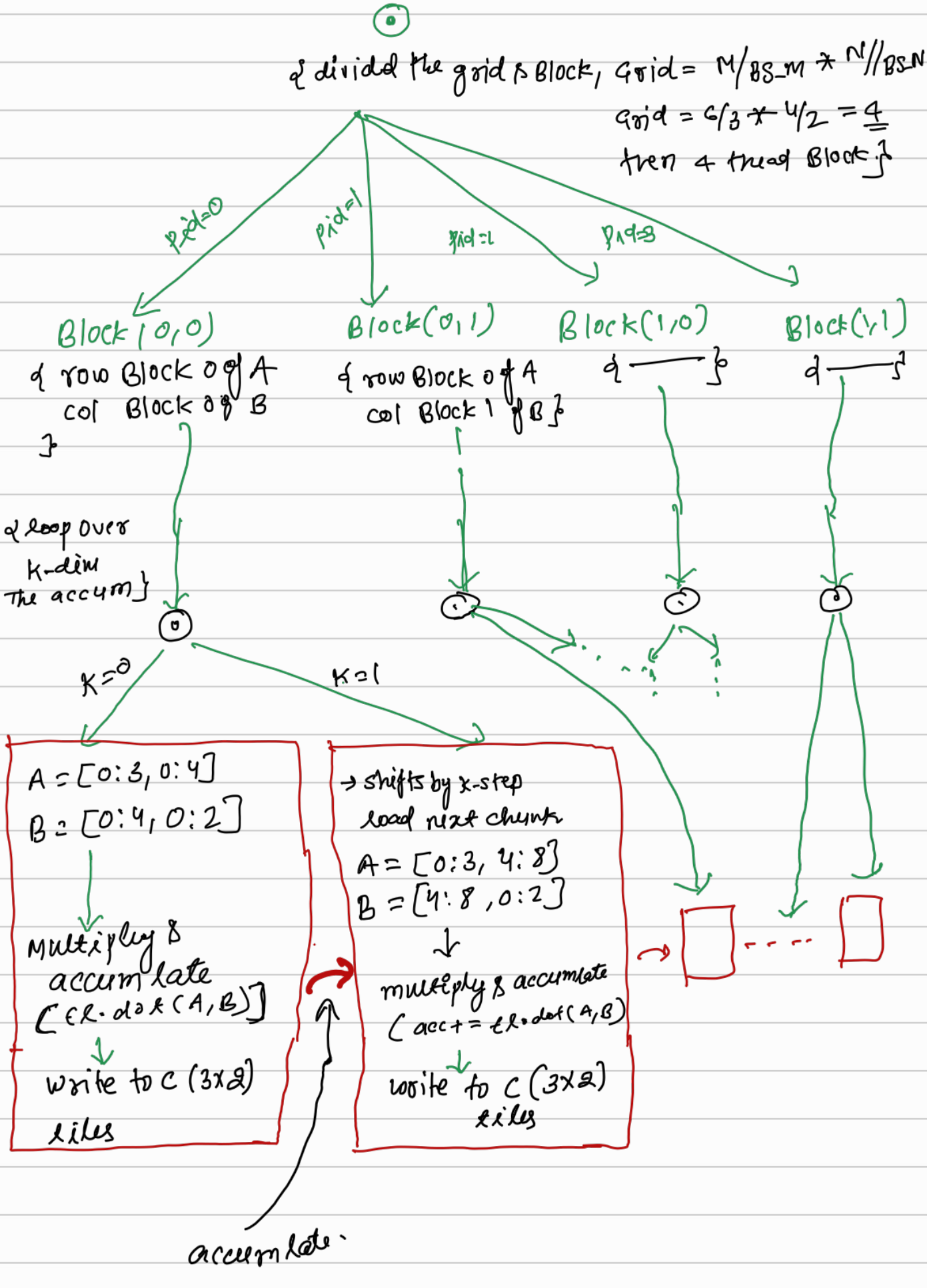
M = 8, N = 4, K = 3



Method 02:-

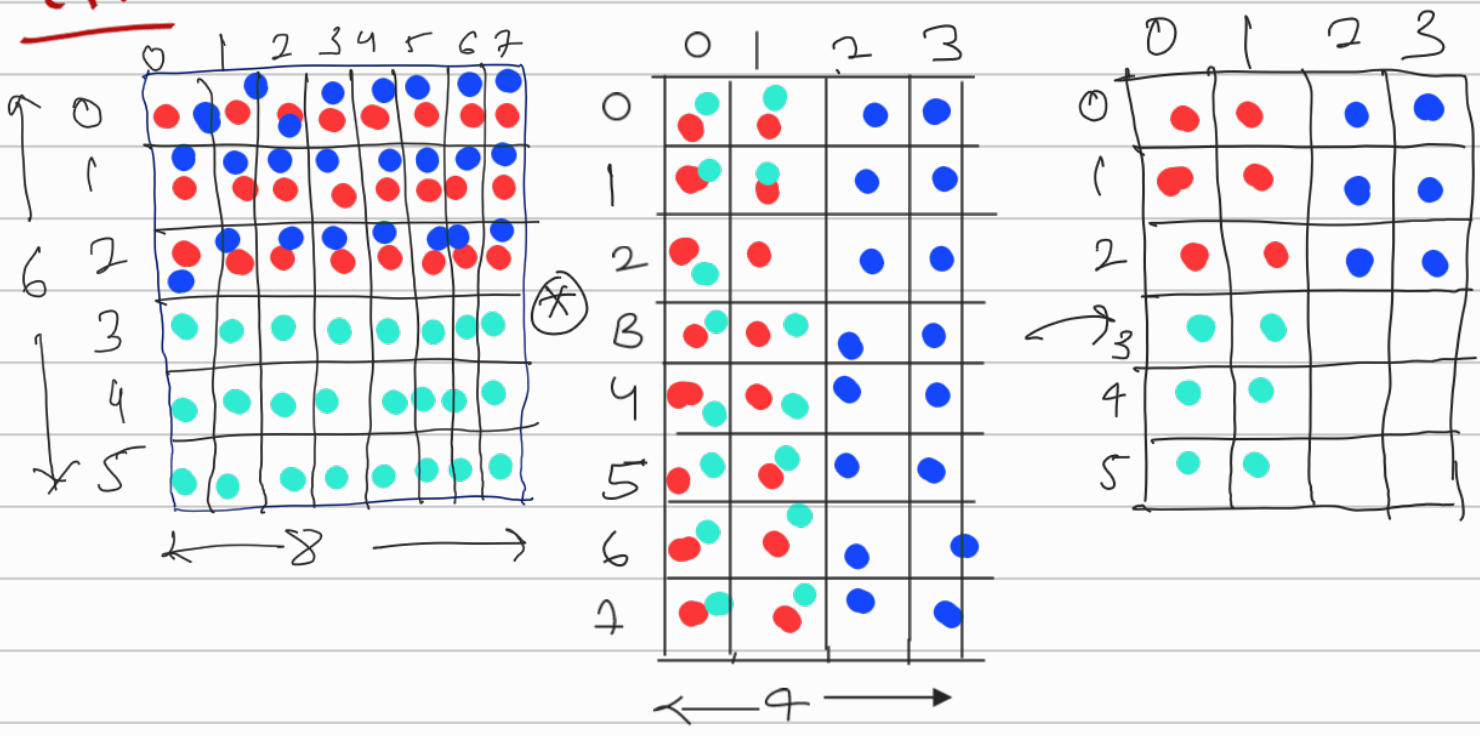
Ex: $A(6\times8) * B(8\times4) \rightarrow C(6\times4)$

Block M=3, N=2, K=4.



04_c (Group Matrix Matmul):-

Ex:-

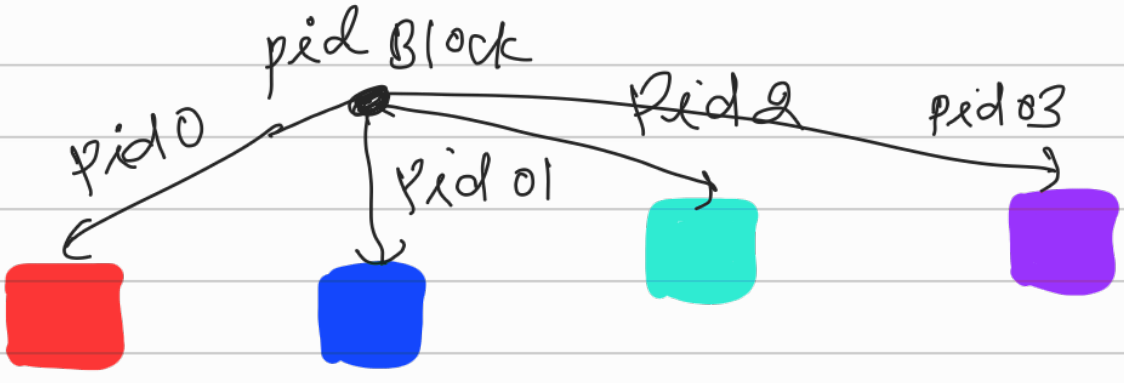


→ 4 pid Block. ∴ Block size_M = 3
Block size_N = 2



$$\Rightarrow \underbrace{\left(\frac{6}{3}\right)}_{A} * \underbrace{\left(\frac{4}{2}\right)}_{B}$$

$$\Rightarrow 2 * 2 \Rightarrow 4$$



→ if memory / matrix size is 10^{10} / high

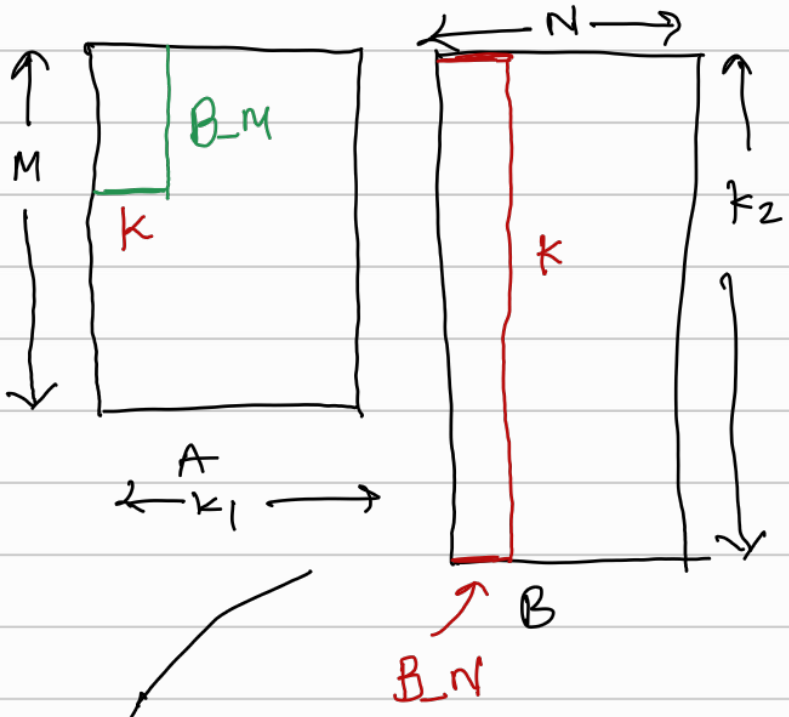
then

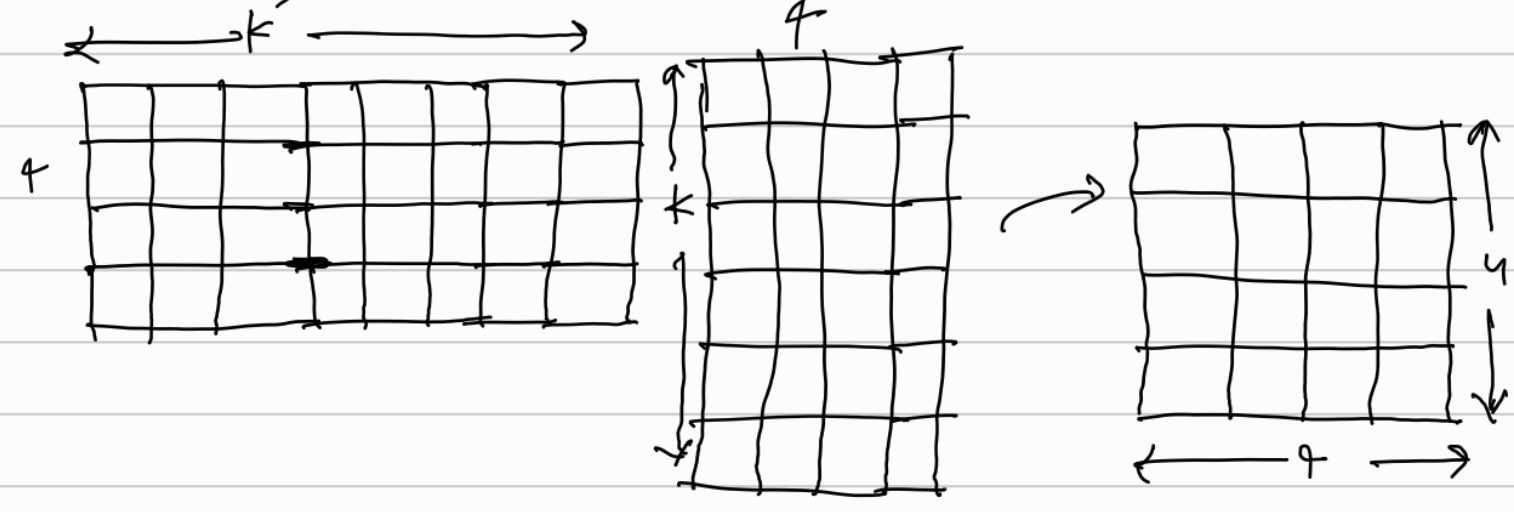
→ Block size_M = 3

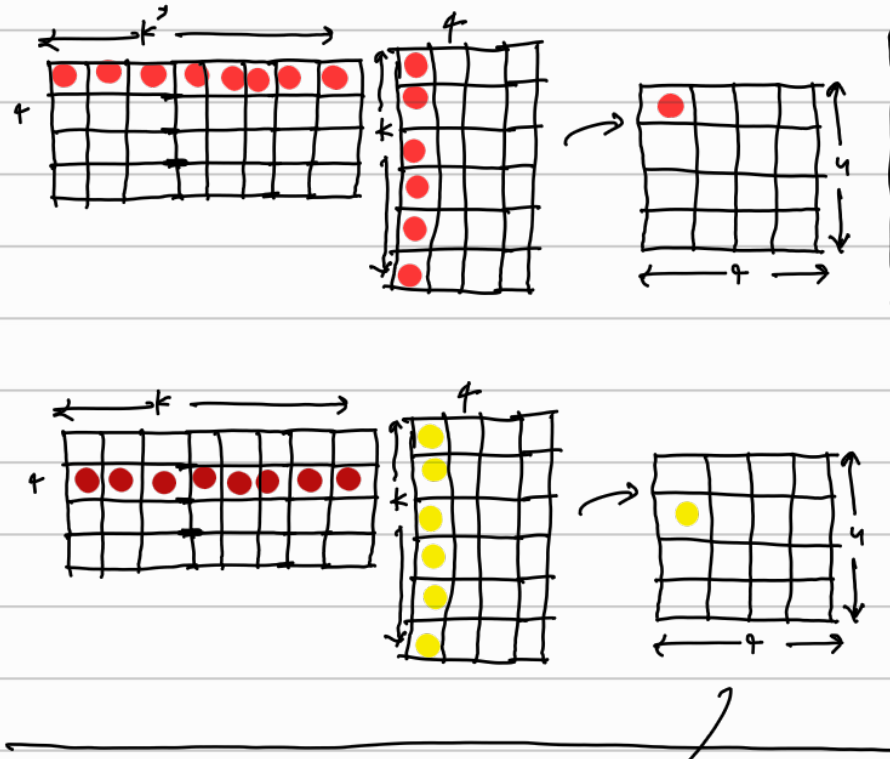
Block size_N = 2

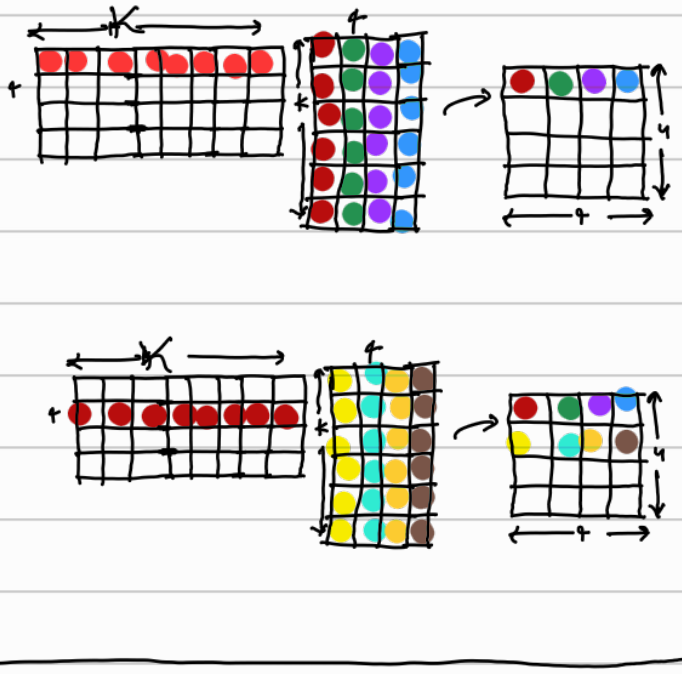
Block size_K = 4 ← add K

Ex:

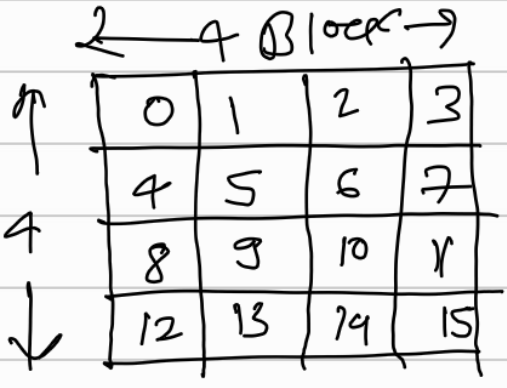


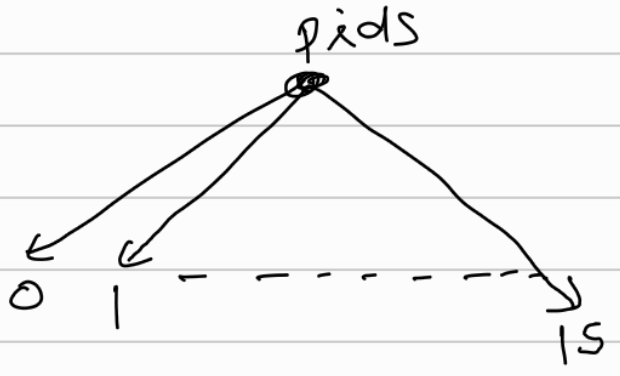




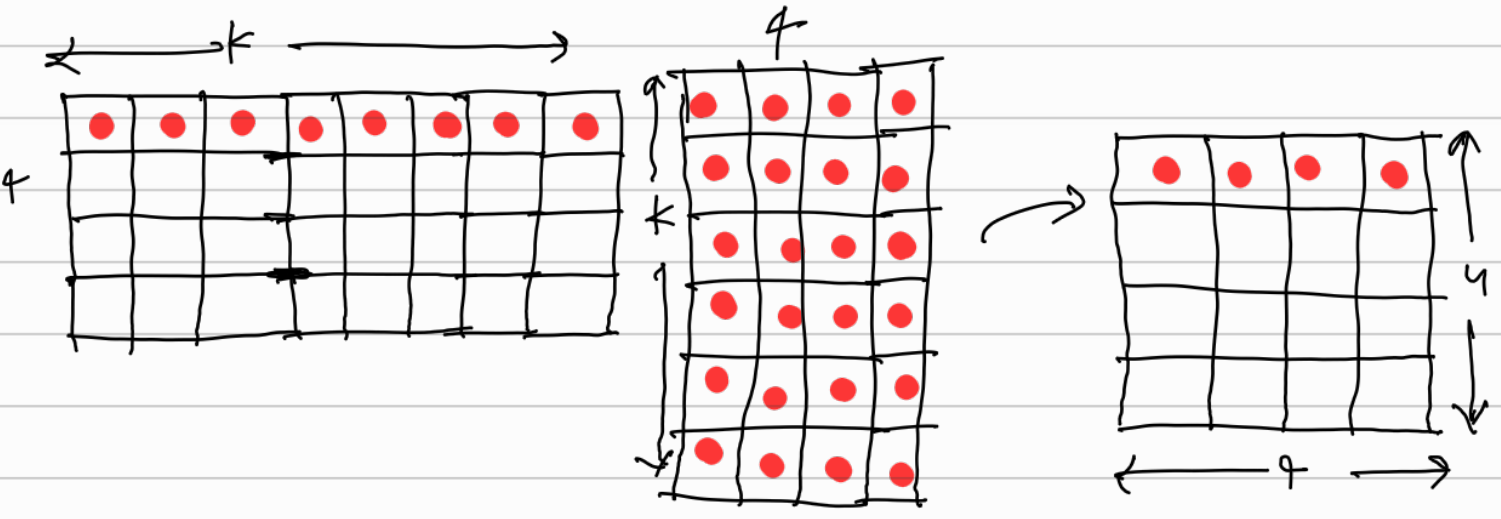


then pids

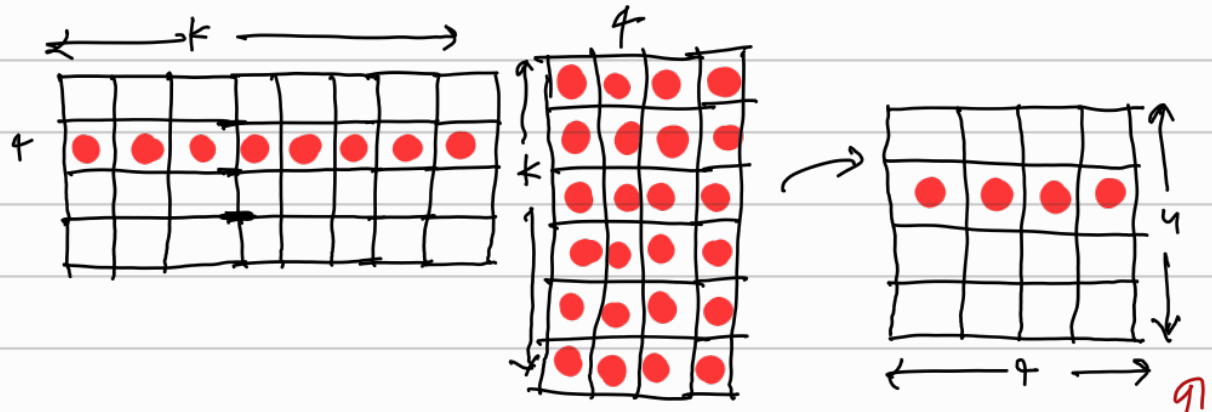




if I make B as group then.

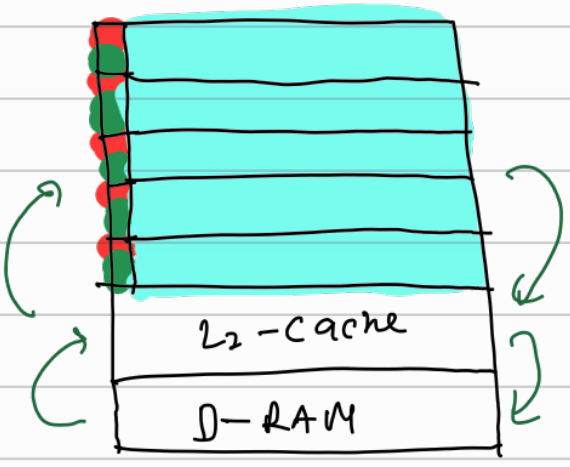


→ $A_{(1\times K)} \cdot B_{(K\times 4)} \longrightarrow C_{(1\times 4)}$ ✓

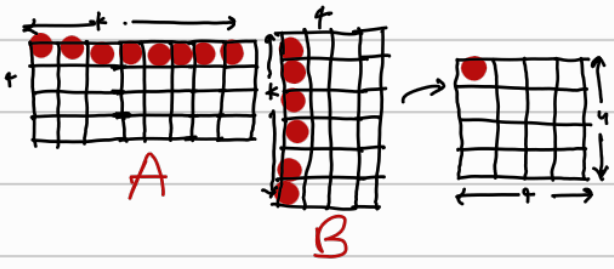


→ $A_{(2 \times k)} * B_{(k \times 4)} \longrightarrow C_{(2 \times 4)}$

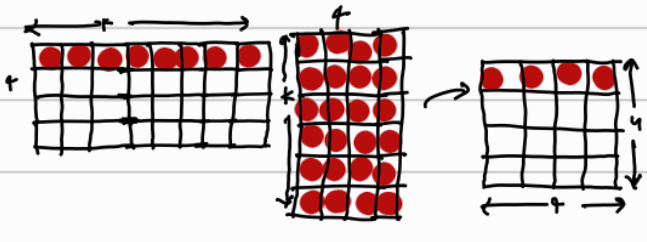
cost with group and without group.



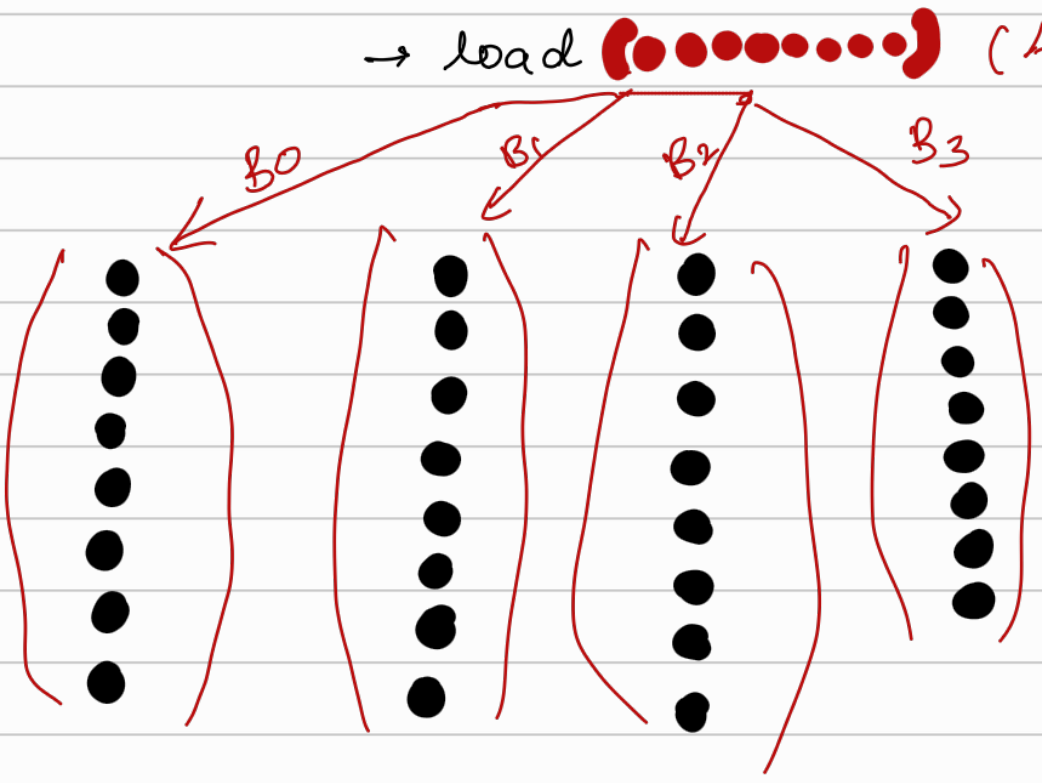
→ without group



→ with group



→ cache Re-use (concept)



→ one Time A row and many time load B all colm

→ then same way A_1 with { B_0 B_1 B_2 B_3 }

→ then same " A_2 with { ——— }

⋮

→ the. ——— An with { B_0 B_1 B_2 B_3 }

→ def group_blocked_matmul_p(A, B, BS_M, BS_N, GroupSize):

M, K = A.shape
K, N = B.shape.

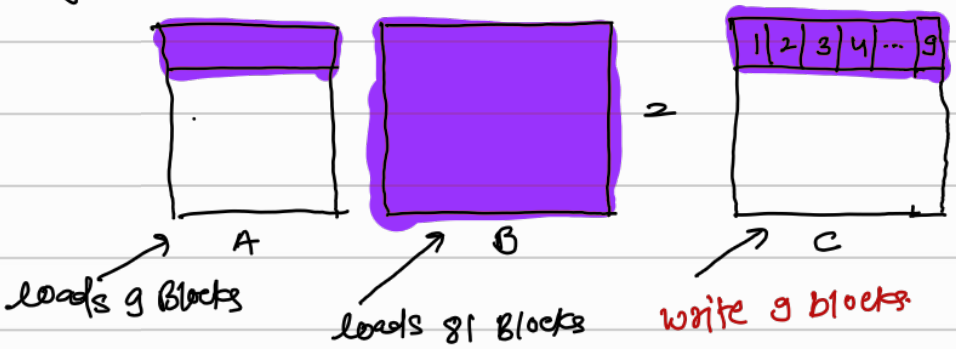
check → M % BS_M == 0
check → N % BS_N == 0
check → BS_M % Groups and BS_N % Groups

num_block_row = M // Block size_M
num_block_cols = N // Block size_N

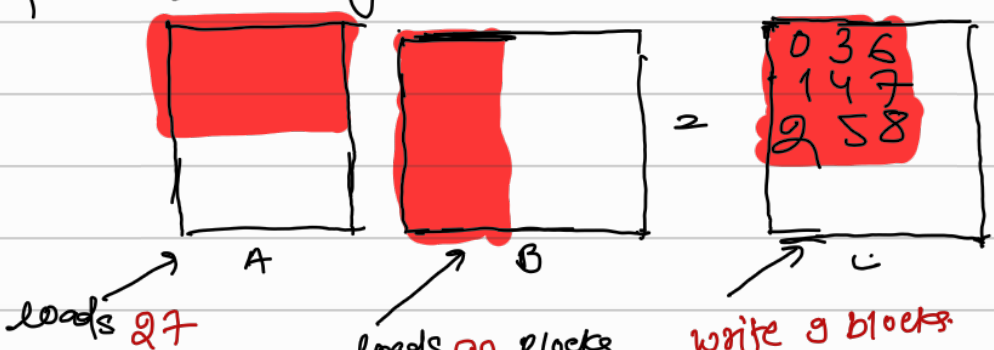
num_pids = list(range(num_blocks_rows * num_blocks_cols))

Example:-

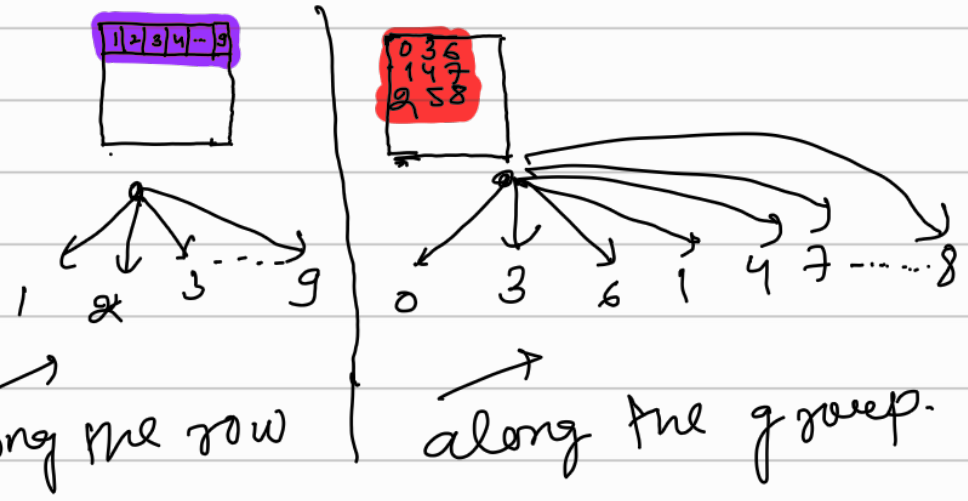
→ Row Major ordering :-



→ Grouped ordering :-



→ pids



→ pid_visual = torch.zeros(num_b_rows, num_b_cols)

→ for i in num_pids:

num_pid_in_g = Groupsize * num_block_cols

which group are we in

groupid = i // num_pid_in_g

first_pid_m = groupid * GROUP_SIZE

group_size_m = min(num_block_rows - first_pid_m, GROUP_SIZE)

what is the groupsize incase not divisible.

where_in_g = i % num_pid_in_g

which_row = where_in_g // group_size_m

pid_m = first_pid_m + which_row

where_in_g = i % num_pid_in_g

pid_n = where_in_g // group_size_m

pid_visual[pid_m, pid_n]

m = pid_m * BLOCK_SIZE_M

$n = pid_n * BLOCK_SIZE_N$

$block_A = A[m:m+BS_M, :]$

$block_B = B[:, n:n+BS_N]$

$C[m:m+BS_M, n:n+BS_N] = Naive_Mat(block_A, block_B)$

return C, pid_visical

Task Row Offset Within Group and Final Row Block Coordinate

Setup Example | A : 6 × 3 | B : 3 × 8 → C : 6 × 8 | BLOCK_M = 3, BLOCK_N = 4 | GROUP_SIZE_M = 2

1. Output Matrix C (6 × 8) Divided into 3 × 4 Tiles

C[0,0] Rows 0 – 2 PID 0	C[0,1] Rows 0 – 2 PID 2
C[1,0] Rows 3 – 5 PID 1	C[1,1] Rows 3 – 5 PID 3

6 rows

8 columns

2. Grouped Ordering (GROUP_SIZE_M = 2)

Group 0 (First 2 Row Blocks)

- PID 0 — Local Row 0 (Row 0–2)
- PID 1 — Local Row 1 (Row 3–5)

Group 1 (Next 2 Row Blocks)

- PID 2 — Local Row 0 (Row 0–2)
- PID 3 — Local Row 1 (Row 3–5)

2 Groups

3. For PID = 0: Step-by-Step Calculation

1. Tasks_Per_Group = group_size_m × num_pid_n = 2 × 2 = 4
2. Position_In_Group = PID % Tasks_Per_Group = 0 % 4 = 0
3. Local_Row = Position_In_Group % group_size_m = 0 % 2 = 0
4. Group_Start_Row = Group_ID × GROUP_SIZE_M = 0 × 2 = 0
5. Final_Target_Row = Group_Start_Row + Local_Row = 0 + 0 = 0

4. What Do These Terms Mean?

Task Row Offset Within Group
→ Which row block inside the current group?
For PID 0: Local_Row = 0

Group 0
PID 0 (Row 0–2)
PID 1 (Row 3–5)

Final Row Block Coordinate
→ Which row block in the whole matrix?
For PID 0: Final_Target_Row = 0

5. Final Result for PID = 0

Selected Output Tile: C[0, 0]
(Rows 0 – 2, Cols 0 – 3)

✓ Final Row Block Coordinate = 0

6. Key Takeaways

- Task row offset within group answers "where am I inside this group?"
- Final row block coordinate answers "where am I in the entire matrix?"
- Grouped ordering improves cache reuse without changing the result.
- Formula: Final_Row = Group_Start_Row + Local_Row

Mathematical Journey of Task PID i = 0 (Grouping 2D Matrix Blocks)

Primary global parameters:

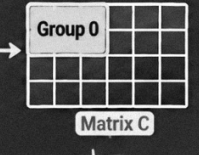
- Matrix A $\in \mathbb{R}^{3\times3}$ partitioned into sub-matrices of BLOCK_SIZE_M = 3, K = 3
- Matrix B $\in \mathbb{R}^{3\times3}$ partitioned into sub-matrices of K = 3 and BLOCK_SIZE_N = 4
- Matrix C $\in \mathbb{R}^{3\times3}$, the target output matrix consisting of four 3 × 4 sub-matrices
- GROUP_SIZE = 2

PID i = 0

THE GREEN TRUNK: Matrix A Pointer Calculation (m_index)

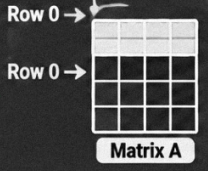
Step-by-step math converts 0 into Row Block indices in A:

Step a: Find Group ID for Output C
Which_Group_ID = i ÷ Tasks_Per_Group = 0 ÷ 4 = 0



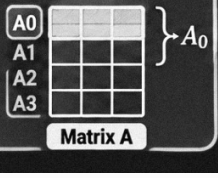
Step b: Translate to Global Matrix Row
Group_Start_Row_y = Which_Group_ID × Block_Height = 0 × 3 = 0

Step c: Task Row Offset within Group
Position_In_Group = i mod Tasks_Per_Group = 0 % 4 = 0
Local_Row = Position_In_Group mod Block_Height = 0 % 3 = 0



Step d: Calculate Final Row Block Coordinate
Final_Target_Row_y = Group_Start_Row_y + Local_Row = 0 + 0 = 0
This matches our row of interest: Block Row 0.

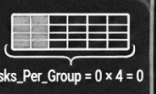
Step e: Map to Memory Start Offset
m = Final_Target_Row_y × BLOCK_SIZE_M = 0 × 3 = 0



THE BLUE TRUNK: Matrix B Pointer Calculation (n_

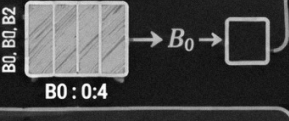
Step-by-step math to locate columns in B:

Step a: Position In Group
Position_In_Group = i mod Tasks_Per_Group = 0 % 4 = 0



Step b: Local Column Block Coordinate
Final_Target_Col_x = Position_In_Group × Block_Width = 0 × 3 = 0

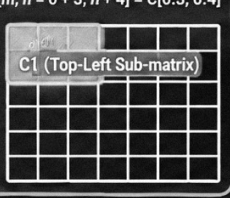
Step c: Convert to Global Memory Columns
n = Final_Target_Col_x × BLOCK_SIZE_N = 0 × 4 = 0



THE ORANGE TRUNK: Output C

Combining results:

Step a: Locate output matrix
C[m, n = 0 + 3, n + 4] = C[0:3, 0:4]



Final Update (Grouped Order)

$\{ A_0 \rightarrow \square \times B_0 \rightarrow C_1 \rightarrow \square \}$

Key Insight:

A massive final visual bracket combines it, mainly visual selects to make the logical hardware grouping. Spatial groupings, and mathematics incredibly clear and intuitive. It. A rich, highly labeled, visual diagram that utilizes correct LaTeX fomatting on a dark background. Free of clutter. No typos.

→ Method 02:- formal Base.

@triton.jit

```
def grouped_matmul_kernel(A_ptr, B_ptr, C_ptr, M, N, K,
                          stride_am, stride_ak,
                          stride_bk, stride_bn,
                          stride_cm, stride_cn,
                          Block_size_M: tl.constexpr,
                          Block_size_N: tl.constexpr,
                          Block_size_K: tl.constexpr,
                          Group_size_M: tl.constexpr):
```

Same Code.

```
pid = tl.program_id(axis=0)
num_pid_m = tl.cdiv(M, BS_M)
num_pid_n = tl.cdiv(N, BS_N)
num_pid_in_group = GS_M * num_pid_n.
group_id = pid // num_pid_in_group
first_pid_m = ____
group_size_m = ____
pid_m = ____
pid_n = ____
```

```
num_blocks_cols = tl.cdiv(N, BLOCK_SIZE_N)
block_m = pid // num_block_cols
block_n = pid % num_block_cols.

offset_m = block_m * BLOCK_size_M + tl.arange(0, BLOCK_SIZE_M)
offset_n = block_n * Block_size_N + tl.arange(0, BLOCK_SIZE_N)
offset_k = tl.arange(0, BLOCK_SIZE_K)
```

```
a_ptrs = A_ptr + (offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak)
b_ptrs = B_ptr + (offs_k[:, None] * stride_bk + offs_n[None, :] * stride_bn)

accumulator = tl.zeros((BS_M, BS_N), dtype=tl.float32)
for k in range(0, tl.cdiv(K, BS_K)):
    a_mask = ((offs_m[:, None] < M) and
              (offs_k[None, :] < K - k * BS_K))
    a = tl.load(a_ptrs, mask=a_mask, other=0.0)

    b_mask = ((offs_k[:, None] < K - k * BS_K) and
              (offs_n[None, :] < N))
    b = tl.load(b_ptrs, mask=b_mask, other=0.0)

    acc += tl.dot(a, b)
    a_ptrs += BS_K * stride_ak
    b_ptrs += BS_K * stride_bk

c = accumulator.to(A_ptr.dtype.element_ty)
c_offsets = stride_cm * offs_m[:, None] +
            stride_cn * offs_n[None, :]
c_mask = ((offs_m[:, None] < M) and
          (offs_n[None, :] < N))

tl.store(c_ptr + c_offsets, c, mask=c_mask)
```